

Aplicação das Normas ISO/IEC 25010 e ISO/IEC/IEEE 15288 — SIDERA PREDICT

Disciplina: Projeto Integrador

Projeto: SIDERA PREDICT — Sistema Inteligente de Inspeção Dimensional

Equipe: Grupo 04

Etapa 1 — Descrição do Produto

(Relacionada ao processo de definição de necessidades da ISO 15288)

Objetivo do Aplicativo

O **SIDERA PREDICT** é um sistema inteligente de inspeção dimensional assistida, desenvolvido para aplicação em ambientes industriais metalúrgicos. O aplicativo utiliza visão computacional baseada em marcadores ChArUco para extrair medidas reais (comprimentos, ângulos, diâmetros, furos, slots) de peças metálicas diretamente no chão de fábrica, com precisão de até 1,5 mm. O operador compara as medidas extraídas com o desenho técnico e registra a conformidade ou não conformidade da peça, com rastreabilidade total e geração de relatórios assistidos por Inteligência Artificial.

Público-Alvo

- **Operadores de Máquina:** Necessitam validar rapidamente a conformidade dimensional das peças sem interromper o fluxo produtivo, utilizando interface simples e otimizada para uso com luvas e por diferentes faixas etárias.
- **Inspetores de Qualidade:** Demandam ferramentas para garantir a conformidade dimensional com rastreabilidade e geração de relatórios exportáveis em PDF e Excel.
- **Gestores Industriais:** Buscam indicadores de desempenho, redução de custos de retrabalho e melhoria contínua dos processos produtivos.

Problema que o Sistema Pretende Resolver

Na indústria metalúrgica, a conformidade dimensional das peças é crítica para a qualidade do produto final. Os principais problemas enfrentados são:

- **Erros dimensionais** frequentes (ângulos fora de especificação, abas fora de medida, geometrias incorretas) que geram retrabalho e devoluções.

- **Inspeção manual e subjetiva**, dependente da atenção do operador e ferramentas como paquímetros, sem padronização.
- **Baixa rastreabilidade**, dificultando a identificação da origem de falhas e do operador responsável.
- **Impacto financeiro direto** por retrabalho, devoluções e insatisfação do cliente.

Visão Geral do Funcionamento do Sistema

O sistema consiste em um aplicativo móvel desenvolvido em **Flutter** (Android/iOS) com as seguintes etapas de operação:

1. **Autenticação:** O operador realiza login com e-mail ou matrícula (campo unificado), com sessão persistente via Supabase Auth.
2. **Captura:** O operador posiciona a peça sobre uma base com 4 marcadores ChArUco e captura a imagem via câmera, com orientação visual de nível (sensor a 90°).
3. **Processamento (Edge AI):** O processamento ocorre localmente no dispositivo via biblioteca C++ nativa (OpenCV), que detecta marcadores, calibra a escala e extrai medidas (arestas, ângulos, furos, semicírculos, slots).
4. **Validação Humana Assistida:** O operador compara as medidas exibidas com o desenho técnico e registra a peça como "Conforme" ou "Não Conforme", com possibilidade de detalhar o motivo de reprovação.
5. **Registro e Relatório IA:** O sistema salva automaticamente a inspeção no Supabase (banco de dados em nuvem), com sincronização offline e geração de relatório técnico via LLM (Ollama). As imagens são armazenadas em bucket privado no Supabase Storage.
6. **Histórico e Exportação:** O operador consulta o histórico de medições e exporta relatórios individuais ou consolidados em PDF e Excel.

Etapa 2 — Definição de Atributos de Qualidade

(Aplicação da ISO/IEC 25010)

1. Segurança (Security)

Descrição: Capacidade do sistema de proteger informações e dados de acessos não autorizados, garantindo confidencialidade, integridade e rastreabilidade.

Justificativa: Em um ambiente industrial, os dados de inspeção são críticos para auditoria e compliance. O SIDERA PREDICT implementa múltiplas camadas de segurança comprovadas no código-fonte:

- **Variáveis de ambiente protegidas (`.env`):** Todas as credenciais sensíveis (URL do Supabase, chave anon, configuração Ollama) são armazenadas em arquivo `.env` que está listado no `.gitignore`, nunca sendo versionado. O arquivo `.env.example` documenta as variáveis necessárias sem expor valores reais. A classe `AppConfig` valida obrigatoriamente todas as variáveis na inicialização (`validateOrThrow()`), impedindo a execução com configuração inválida.
- **Validação rigorosa de URLs:** O `AppConfig` implementa validação que rejeita URLs com `localhost`, IPs internos (10.x, 127.x, 169.254.x, 172.16-31.x, 192.168.x), domínios `.local` / `.lan`, e exige HTTPS para endpoints externos (Ollama), prevenindo exposição acidental de dados em redes inseguras.
- **Autenticação obrigatória via Supabase Auth:** O sistema exige autenticação completa do funcionário (e-mail ou matrícula + senha). O `SplashViewModel` verifica a sessão ativa e redireciona para login caso não haja sessão válida. A unicidade de e-mail e matrícula é validada via funções PostgreSQL `security definer` no banco de dados.
- **Row Level Security (RLS):** O schema SQL aplica RLS em todas as tabelas (`profiles`, `measurement_records`) e no Storage. Cada usuário só pode ler/modificar/excluir seus próprios registros e imagens. As policies de Storage validam que o `foldername` do path corresponda ao `auth.uid()`.
- **Bucket privado de imagens:** O bucket `measurement-images` é configurado como `public: false`, com limite de 20MB e tipos MIME restritos a `image/jpeg` e `image/png`.
- **Validação de propriedade em toda operação:** O `SupabaseMeasurementService` e o `SupabaseImageStorageService` verificam se o `ownerUserId` do registro corresponde ao usuário autenticado antes de salvar ou enviar imagens, lançando `StateError` se a sessão foi alterada.

2. Confiabilidade (Reliability)

Descrição: Capacidade do sistema de desempenhar suas funções sob condições definidas, por um período de tempo definido, com tolerância a falhas e recuperabilidade.

Justificativa: O aplicativo opera em chão de fábrica onde a conectividade é instável e falhas não podem interromper a produção. O sistema implementa:

- **Arquitetura Offline-First:** O `MeasurementRepository` salva primeiro localmente (`LocalHistoryStore`) e sincroniza com o Supabase em segundo plano. Se a conexão falhar, os registros permanecem locais e são sincronizados automaticamente via timer periódico (`_startSyncTimer`, a cada 45 segundos).
- **Fila de deleções pendentes:** As exclusões que falham por falta de conectividade são armazenadas em `SharedPreferences` (`_pendingDeletions`) e reprocessadas na próxima sincronização.
- **Retry com backoff exponencial para IA:** Se a geração de relatório falha, o sistema agenda retentativas com delays progressivos: 2s → 10s → 30s → 60s → 120s → 300s

(`_retryDelayForAttempt`), sem bloquear a interface.

- **Lock sequencial de processamento IA:** O `_aiProcessingLock` (Completer) garante que apenas um relatório IA é processado por vez, evitando sobrecarga do servidor e condições de corrida.
- **Controle de sessão via epoch:** O `_sessionEpoch` é incrementado a cada mudança de autenticação, descartando automaticamente todas as operações em background da sessão anterior, prevenindo inconsistências.
- **Cache em disco e memória para imagens:** O `SupabaseImageStorageService` implementa cache em dois níveis (`_memoryCache` em Map e disco em `image_cache/`), reduzindo dependência de rede e acelerando a exibição do histórico.
- **Reconciliação local/remota:** O método `reconcileRemoteSnapshot` faz merge inteligente entre registros locais e remotos, preservando relatórios IA completos localmente mesmo se o remoto ainda estiver pendente.

3. Usabilidade (Usability)

Descrição: Capacidade do sistema de ser compreendido, aprendido e operado com eficácia por diferentes perfis de usuário.

Justificativa: O público-alvo inclui operadores com pouca experiência em tecnologia, que trabalham com luvas em ambiente industrial. O sistema implementa:

- **Fluxo de cadastro em etapas (step-by-step):** O `SignupViewModel` guia o operador por 4 etapas (Nome → Matrícula → E-mail → Senha) com validação em tempo real a cada passo. A disponibilidade de matrícula e e-mail é verificada antes de avançar para o próximo passo.
- **Login unificado com detecção automática:** O campo de identificação aceita tanto e-mail quanto matrícula. O `AuthService.signIn` detecta automaticamente se o input contém @ para determinar o tipo de identificador, eliminando seletores de tipo.
- **Validação visual de senha em tempo real:** O `PasswordRequirementsWidget` e `isPasswordStrong` verificam 4 critérios (8+ caracteres, maiúscula, número, caractere especial) com feedback visual imediato.
- **Overlay de sucesso animado:** O `AuthSuccessOverlay` fornece feedback visual imediato após login/cadastro bem-sucedido, com animação suave antes de redirecionar.
- **Alertas glassmorphism animados:** O `AppAlerts` exibe notificações com efeito blur (`BackdropFilter`), animação `elasticOut` e auto-dismiss após 4 segundos.
- **Temas de acessibilidade:** O `SettingsService` oferece Modo Escuro e Modo Alto Contraste (com cores amarelo/preto para baixa visão), gerenciados pelo `SettingsViewModel` com persistência via `SharedPreferences` .
- **Nomeação automática de peças:** O `buildAutomaticPieceName` gera nomes no formato "Peça N - DD/MM/AAAA - Nome do Funcionário", evitando conflitos entre operadores.

4. Eficiência de Desempenho (Performance Efficiency)

Descrição: Capacidade do sistema de fornecer desempenho adequado relativo à quantidade de recursos utilizados sob condições estabelecidas.

Justificativa: A inspeção não pode impactar o ritmo produtivo. O sistema atende ao requisito de latência máxima de 2 segundos entre captura e resultado:

- **Processamento em Isolate:** O `NativeVisionBridge.analyze()` executa o processamento OpenCV em `Isolate.run()`, mantendo a UI responsiva durante a análise da imagem.
- **Biblioteca nativa C++ (FFI):** O motor de visão computacional (`libvision_engine.so`) é compilado em C++ nativo e acessado via `dart:ffi`, garantindo desempenho otimizado sem overhead de interpretação.
- **Otimização de imagens em background:** O `MeasurementRepository` usa `compute()` (isolate) para redimensionar imagens (`_optimizeImageTask : max 300px, JPEG quality 60, max 50KB para thumbnails`), evitando bloqueio da thread principal.
- **Pré-processamento de orientação EXIF:** O `_prepareInputImage` normaliza a orientação da imagem (`bakeOrientation`) antes de enviar ao OpenCV, eliminando processamento redundante no motor nativo.
- **Streaming de relatório IA:** O `ollamaReportService.streamReport()` recebe tokens em tempo real via HTTP streaming (`LineSplitter`), exibindo o relatório progressivamente ao invés de esperar a geração completa.
- **Cache duplo (memória + disco):** Imagens baixadas do Supabase são cacheadas em memória (`Map`) e em disco (`getApplicationSupportDirectory`), eliminando downloads repetidos.

5. Manutenibilidade (Maintainability)

Descrição: Capacidade do sistema de ser modificado, corrigido, melhorado ou adaptado com facilidade.

Justificativa: O projeto precisa evoluir continuamente com novos tipos de geometria, integrações e funcionalidades. A arquitetura demonstra:

- **Arquitetura MVVM consistente:** Cada feature (auth, inspection, reports, menu, settings, splash) segue o padrão Model-View-ViewModel com separação clara em diretórios `model/`, `view/`, `viewmodel/`, `data/`.
- **Injeção de dependências:** Todos os serviços são injetados via construtor. O `SideraPredictApp` constrói a árvore de dependências e usa `MultiProvider` + `ChangeNotifierProvider` para distribuí-las.
- **Testabilidade via fakes:** O diretório `test/fakes/` contém `FakeAuthService`, `FakeMeasurementRepository` e `FakeMeasurementService`, permitindo testes unitários sem dependência de Supabase real.
- **Padrão Repository:** O `MeasurementRepository` abstrai toda a lógica de persistência (local + remota + imagens + IA), isolando os ViewModels do conhecimento sobre storage.

- **Configuração centralizada:** O `AppConfig` centraliza toda configuração de ambiente, com validação rigorosa e mensagens de erro descritivas em português.
 - **Schema SQL versionado:** O arquivo `supabase/schema.sql` contém toda a estrutura do banco de dados com `CREATE IF NOT EXISTS`, `DROP POLICY IF EXISTS` e tratamento de conflitos, permitindo execução idempotente.
-

Etapa 3 — Definição de Requisitos de Qualidade

Segurança

- **RQ-S01:** O sistema não deve armazenar credenciais de acesso ao backend (URLs, chaves API) no código-fonte versionado; todas devem ser carregadas via variáveis de ambiente (`.env`) excluídas do controle de versão.
- **RQ-S02:** O sistema deve validar obrigatoriamente todas as variáveis de ambiente na inicialização, impedindo a execução com configuração incompleta ou inválida.
- **RQ-S03:** O sistema deve rejeitar URLs de endpoints externos que apontem para endereços privados (`localhost` , IPs internos) ou utilizem protocolo HTTP não seguro.
- **RQ-S04:** Cada operação de escrita no banco de dados deve verificar que o usuário autenticado é o proprietário do registro, impedindo modificações cruzadas.
- **RQ-S05:** O sistema deve garantir a unicidade de e-mail e matrícula no cadastro, validando via funções PostgreSQL `security definer` antes da criação da conta.

Confiabilidade

- **RQ-C01:** O sistema deve persistir medições localmente antes de qualquer tentativa de sincronização remota, garantindo que nenhum dado seja perdido por falha de rede.
- **RQ-C02:** Operações de exclusão que falharem por indisponibilidade de rede devem ser enfileiradas e reprocessadas automaticamente na próxima sincronização.
- **RQ-C03:** Falhas na geração de relatório IA devem ser tratadas com retentativas automáticas com backoff progressivo (2s a 5min), sem intervenção do operador.
- **RQ-C04:** O sistema deve descartar automaticamente todas as operações em background de uma sessão anterior quando o usuário trocar de conta.

Usabilidade

- **RQ-U01:** O cadastro deve ser realizado em etapas guiadas, com no máximo um campo por etapa e validação visual imediata antes de avançar.
- **RQ-U02:** O login deve aceitar e-mail ou matrícula em um único campo, detectando automaticamente o tipo de identificador sem seletores.

- **RQ-U03:** A força da senha deve ser validada visualmente em tempo real, exibindo indicadores para cada critério (comprimento, maiúscula, número, caractere especial).
- **RQ-U04:** O sistema deve oferecer no mínimo três modos visuais (Claro, Escuro, Alto Contraste) com persistência da preferência do usuário.

Eficiência de Desempenho

- **RQ-P01:** O processamento de visão computacional deve ser executado em thread separada (Isolate), sem bloquear a interface do usuário.
- **RQ-P02:** O motor de visão computacional deve ser implementado em código nativo (C/C++), acessado via FFI, para garantir desempenho adequado em dispositivos móveis.
- **RQ-P03:** Imagens de inspeção devem ser cacheadas em dois níveis (memória e disco), evitando downloads repetidos do armazenamento em nuvem.
- **RQ-P04:** Relatórios de IA devem ser exibidos progressivamente via streaming, sem bloquear a interface até a geração completa.

Manutenibilidade

- **RQ-M01:** O código-fonte deve seguir o padrão arquitetural MVVM, com separação obrigatória entre camadas de dados, lógica de negócio e interface.
- **RQ-M02:** Todos os serviços de infraestrutura devem aceitar injeção de dependências via construtor, permitindo substituição por implementações de teste.
- **RQ-M03:** O schema do banco de dados deve ser versionado em arquivo SQL com operações idempotentes (IF NOT EXISTS, ON CONFLICT).

Etapa 4 — Decisões de Engenharia

Arquitetura do Sistema

Decisão	Justificativa	Atributo de Qualidade
MVVM com Provider	Separação clara entre View, ViewModel e Model. Os ViewModels (<code>InspectionViewModel</code> , <code>AuthViewModel</code> , <code>SettingsViewModel</code>) encapsulam a lógica de negócio e notificam a UI via <code>ChangeNotifier</code> . O <code>MultiProvider</code> na raiz distribui dependências.	Manutenibilidade
Feature-based directory structure	Cada feature (auth, inspection, reports, menu, settings, splash) tem seus próprios diretórios <code>view/</code> , <code>viewmodel/</code> , <code>model/</code> , <code>data/</code> , facilitando a localização e evolução independente de funcionalidades.	Manutenibilidade

Decisão	Justificativa	Atributo de Qualidade
Repository Pattern	O <code>MeasurementRepository</code> centraliza toda lógica de persistência, abstraindo a complexidade de sincronização local/remota, upload de imagens e geração de IA dos ViewModels.	Manutenibilidade, Confiabilidade

Organização do Código e Segurança

Decisão	Justificativa	Atributo de Qualidade
Variáveis de ambiente via <code>flutter_dotenv</code>	Credenciais carregadas de <code>.env</code> (excluído do Git). O <code>AppConfig.validateOrThrow()</code> impede execução com configuração inválida. O <code>.env.example</code> documenta as variáveis sem expor valores.	Segurança
Validação de hosts privados no <code>AppConfig</code>	Método <code>_isPrivateHost()</code> rejeita localhost, IPs RFC 1918, domínios <code>.local/.lan</code> , garantindo que endpoints de IA apontem apenas para URLs públicas HTTPS.	Segurança
Row Level Security (RLS) no Supabase	Policies SQL garantem que cada usuário só acessa seus próprios perfis, medições e imagens. Funções <code>security_definer (email_for_matricula, is_matricula_available, is_email_available)</code> encapsulam queries sensíveis.	Segurança
Bucket de imagens privado	Configurado como <code>public: false</code> , com MIME types restritos e policies de Storage que validam <code>auth.uid()</code> no path da imagem.	Segurança
Verificação de propriedade em toda operação	<code>SupabaseMeasurementService.saveRecord()</code> e <code>SupabaseImageStorageService.uploadMeasurementImages()</code> comparam <code>ownerUserId</code> com o usuário autenticado antes de executar.	Segurança

Estrutura de Dados

Decisão	Justificativa	Atributo de Qualidade
Payload JSONB no PostgreSQL	A tabela <code>measurement_records</code> armazena as medições como <code>payload jsonb</code> , permitindo evolução do schema de medições sem migrações de banco de dados. Índices em <code>(user_id, created_at desc)</code> otimizam consultas.	Manutenibilidade, Eficiência

Decisão	Justificativa	Atributo de Qualidade
Modelo imutável com <code>copyWith</code>	<code>MeasurementRecord</code> e <code>MeasurementDraft</code> são essencialmente imutáveis, usando <code>copyWith</code> para atualizações, prevenindo efeitos colaterais e facilitando testes.	Manutenibilidade, Confiabilidade
Enum tipado para segmentos de peça	<code>PieceSegmentType</code> (edge, semicircle, hole, angle, slot, chamfer, diameter, spacing) com métodos <code>storageValue / fromStorage</code> garante serialização segura e extensibilidade.	Manutenibilidade
Separação de draft e record	<code>MeasurementDraft</code> (dados da medição bruta) é separado de <code>MeasurementRecord</code> (registro completo com metadata), permitindo evolução independente.	Manutenibilidade

Definição de Interface e Acessibilidade

Decisão	Justificativa	Atributo de Qualidade
Três temas (Light, Dark, High Contrast)	<code>buildLightTheme()</code> , <code>buildDarkTheme()</code> e <code>buildHighContrastTheme()</code> com paletas distintas. O modo Alto Contraste usa amarelo/preto (dark) ou preto/branco (light) para acessibilidade de baixa visão.	Usabilidade
Cadastro em etapas com <code>PageController</code>	O <code>SignupViewModel</code> gerencia 4 etapas com animações <code>easeOutCubic</code> de 300ms, validando cada campo antes de avançar e verificando duplicidade de matrícula/e-mail no servidor.	Usabilidade
Alertas overlay com glassmorphism	<code>_TopAlertWidget</code> usa <code>BackdropFilter</code> com blur, animação <code>elasticOut</code> e auto-dismiss, fornecendo feedback não-intrusivo adequado para ambiente industrial.	Usabilidade
Login unificado com detecção automática	<code>AuthService.signIn()</code> verifica se o input contém <code>@</code> para decidir entre login por e-mail ou por matrícula (via RPC <code>email_for_matricula</code>), eliminando complexidade para o operador.	Usabilidade

Processamento e Desempenho

Decisão	Justificativa	Atributo de Qualidade
Motor de visão em C++ nativo via FFI	A <code>libvision_engine.so</code> (97KB de código C++) processa detecção de marcadores ChArUco e extração de geometria. Acessada via <code>dart:ffi</code> com structs nativos (<code>MeasurementResultStruct</code>).	Eficiência de Desempenho
Processamento em Isolate	<code>NativeVisionBridge.analyze()</code> executa em <code>Isolate.run()</code> , mantendo a UI a 60fps durante processamento intensivo de imagem.	Eficiência de Desempenho
Otimização de imagens em <code>compute()</code>	<code>_optimizeImageTask</code> redimensiona para thumbnails (300px, JPEG Q60, max 50KB) em isolate separado, sem bloquear a thread principal.	Eficiência de Desempenho
Streaming SSE para relatórios IA	<code>OllamaReportService.streamReport()</code> recebe tokens em tempo real via HTTP streaming com <code>LineSplitter</code> , exibindo texto progressivamente. Timeout de 60s para conexão e 15s para inatividade de stream.	Eficiência de Desempenho

Confiabilidade e Tolerância a Falhas

Decisão	Justificativa	Atributo de Qualidade
Arquitetura Offline-First	<code>LocalHistoryStore</code> persiste em JSON local por sessão. O <code>MeasurementRepository</code> salva localmente primeiro e sincroniza em background com retry automático a cada 45 segundos.	Confiabilidade
Fila de deleções pendentes	<code>_pendingDeletions</code> é persistida em <code>SharedPreferences</code> e reprocessada no próximo <code>loadHistory()</code> , garantindo consistência eventual.	Confiabilidade
Retry com backoff para IA	Schedule progressivo (2s, 10s, 30s, 60s, 120s, 300s) com <code>Timer</code> individual por registro, limitando carga no servidor.	Confiabilidade
Session epoch para isolamento	<code>_sessionEpoch</code> descarta operações de sessões anteriores, prevenindo inconsistências em troca rápida de contas.	Confiabilidade
Reconciliação merge inteligente	<code>mergeRemoteRecordWithLocalFallback</code> preserva relatórios IA completos localmente, paths de imagem locais e metadados, resolvendo conflitos local/remoto de forma determinística.	Confiabilidade

Resumo das Evidências no Código-Fonte

Atributo ISO 25010	Principais Evidências no Código
Segurança	<code>.env</code> + <code>.gitignore</code> , <code>AppConfig.validateOrThrow()</code> , <code>_isPrivateHost()</code> , RLS no <code>schema.sql</code> , bucket privado, verificação de <code>ownerUserId</code>
Confiabilidade	<code>LocalHistoryStore</code> , <code>_pendingDeletions</code> , <code>_retryDelayForAttempt()</code> , <code>_sessionEpoch</code> , <code>reconcileRemoteSnapshot()</code> , <code>_startSyncTimer()</code>
Usabilidade	<code>SignupViewModel</code> (4 etapas), login unificado, <code>PasswordRequirementsWidget</code> , <code>buildHighContrastTheme()</code> , <code>AppAlerts</code> <code>glassmorphism</code>
Eficiência	<code>Isolate.run()</code> , <code>dart:ffi</code> , <code>compute()</code> , <code>streamReport()</code> , <code>cache</code> memória+disco
Manutenibilidade	MVVM, Repository Pattern, <code>test/fakes/</code> , injeção via construtor, <code>schema.sql</code> idempotente

